Trabalho Prático 2: Pipeline RISC-V

Grupo: Arthur Signorini, Cauã Costa, Otavio Augusto, Henrique Carvalho, Laura Persilva

1. Introdução

Este relatório detalha a implementação e análise de um pipeline RISC-V de 5 estágios, conforme os requisitos do Trabalho Prático 2. O objetivo principal é aprofundar a compreensão dos mecanismos de execução em pipeline, com foco na identificação e resolução de hazards de dados (RAW) e controle (branches), bem como na avaliação do impacto dessas soluções no desempenho do processador. O projeto base fornecido, didático e não sintetizável, serviu como plataforma para a aplicação prática dos conceitos teóricos.

2. Objetivos

Os objetivos deste trabalho prático foram:

- Compreender a execução sobreposta de instruções em um pipeline.
- Identificar e tratar hazards de dados (RAW).
- Implementar técnicas de resolução de dependências, como o forwarding.
- Gerenciar a inserção de bolhas (stalls) no pipeline.
- Tratar hazards de controle causados por branches.
- Realizar o flush de pipeline quando necessário.
- Analisar o impacto das implementações no desempenho, medido pelo CPI (Cycles Per Instruction).

3. Escopo do Trabalho

O trabalho foi desenvolvido sobre um processador RISC-V simplificado, modelado a partir da Figura e4.14.3 do Capítulo 4 do livro *Computer Organization and Design RISC-*

V Edition (Patterson & Hennessy). O foco foi estritamente didático, visando a compreensão dos princípios do pipeline, e não a construção de um processador real ou a implementação de um ISA completo.

4. Ambiente de Simulação

O ambiente de simulação utilizado foi baseado em Verilog, com as seguintes ferramentas:

- **Icarus Verilog:** Compilador e simulador para código Verilog.
- **GTKWave:** Visualizador de waveforms para análise detalhada dos sinais do circuito.

O projeto incluiu um testbench (`tb_RISCVCPU.v`) que permitiu a execução de programas de teste e a coleta de métricas de desempenho, como ciclos, instruções, stalls, bypasses, branches e flushes.

5. Estrutura do Trabalho (Partes Incrementais)

5.1. Parte 1: Branch e Controle de Fluxo

Nesta primeira etapa, o foco foi na implementação do desvio condicional (`BEQ`) e no tratamento dos hazards de controle.

Implementação: O módulo `BranchUnit.v` foi modificado para incluir a lógica da instrução `BEQ`. A implementação envolveu:

1. **Comparação de Operandos:** Os valores dos registradores `rs1` e `rs2` foram comparados no estágio EX. Se fossem iguais, o desvio era tomado.
2. **Cálculo do Endereço de Destino:** O `branch_target` foi calculado somando o `pc_ex` (Program Counter no estágio EX) com o `branch_imm` (imediato de desvio).
3. **Atualização do PC e Flush:** O sinal `branch_taken` foi ativado quando o desvio era tomado. No módulo `RISCVCPU.v`, este sinal é utilizado para atualizar o PC com o `branch_target` e para realizar o flush das instruções que já haviam sido carregadas nos estágios IF/ID e ID/EX, substituindo-as por NOPs.

Resultados da Simulação (Após Parte 1):

Métrica	Valor Inicial	Após Parte 1
Cycles	14	14
Instructions	9	7
Stalls	0	0
Bypasses	0	0
Branches	0	1
Flushes	0	1
CPI	1.56	2.00
Erros	6	5

Análise: Após a implementação da `BranchUnit`, o processador passou a detectar e executar corretamente a instrução `BEQ`. O `branch_taken` foi ativado, resultando em um flush de 1 instrução. A falha no registrador `x5` (que deveria ser descartado) foi resolvida, indicando que o mecanismo de flush funcionou. No entanto, o CPI aumentou de 1.56 para 2.00. Isso reflete a penalidade de branch, onde ciclos são perdidos devido ao descarte de instruções especulativas e à inserção de NOPs.

5.2. Parte 2: Executando o Programa de Teste (via Software)

Nesta etapa, o objetivo foi fazer o programa de teste fornecido (`load_program_full_dependencies`) executar corretamente sem a necessidade de forwarding ou hazard detection no hardware.

Implementação: O arquivo `tb_RISCVCPU.v` foi modificado para inserir NOPs (No Operation) entre instruções que apresentavam dependências de dados. Para garantir que o valor correto estivesse disponível, foram inseridos 3 NOPs entre as instruções dependentes. Além disso, a inserção de NOPs alterou os endereços das instruções, exigindo o recálculo e ajuste do offset da instrução `BEQ label`.

Resultados da Simulação (Após Parte 2):

Métrica	Após Parte 1	Após Parte 2
Cycles	14	26
Instructions	7	7
Stalls	0	0
Bypasses	0	0
Branches	1	1
Flushes	1	1
CPI	2.00	3.71
Erros	5	0 (PASS)

Análise: Com a inserção manual dos NOPs e o ajuste do offset do `BEQ`, o programa de teste passou a executar corretamente. No entanto, o custo foi um aumento significativo no CPI, que saltou para 3.71. Este valor elevado demonstra a ineficiência de resolver hazards de dados exclusivamente via software.

5.3. Parte 3: Hazard Detection e Forwarding (via Hardware)

Esta etapa final consistiu na implementação das unidades de Forwarding e Hazard Detection no hardware.

Implementação:

1. **ForwardingUnit.v:** Implementado para detectar dependências de dados e encaminhar os resultados de estágios posteriores para estágios anteriores. A lógica priorizou o forwarding do estágio MEM e, em seguida, do estágio WB.
2. **HazardDetectionUnit.v:** Implementado para detectar hazards do tipo load-use. Quando uma instrução `LW` está no estágio EX/MEM e a seguinte tenta usar seu destino, um stall de um ciclo é inserido.

Resultados da Simulação (Final):

Métrica	Após Parte 2	Após Parte 3
Cycles	26	15
Instructions	7	7
Stalls	0	1
Bypasses	0	5
Branches	1	1
Flushes	1	1
CPI	3.71	2.14
Erros	0 (PASS)	0 (PASS)

Análise: O desempenho melhorou drasticamente, com o CPI caindo para 2.14. A ocorrência de 1 stall é correta (hazard load-use). Foram registrados 5 bypasses, indicando que o forwarding resolveu com sucesso as dependências de dados.

6. Análise Esperada (Geral)

- **Por que Stalls Ocorrem:** São inseridos para resolver hazards que o forwarding não contorna, como o load-use.
- **Quando Forwarding Resolve Dependências:** Eficaz quando o resultado está disponível em estágios posteriores e pode ser encaminhado para a entrada da ALU.
- **Impacto do Branch no Desempenho:** Causa penalidade devido ao flush de instruções no caminho errado.

7. Entrega

A entrega consiste neste relatório e um link para o repositório GitHub contendo as implementações nos módulos `BranchUnit.v`, `ForwardingUnit.v` e `HazardDetectionUnit.v`.

8. Uso de Inteligência Artificial (IA)

Este trabalho foi desenvolvido com o auxílio de Inteligência Artificial (Manus AI), seguindo as diretrizes de transparência e responsabilidade técnica. A abordagem seguiu o modelo de **Spec-Driven Development (SDD)**.

8.1 Transparência e Metodologia

A IA foi utilizada para:

- **Geração de Código Verilog:** Implementação da lógica de controle.
- **Depuração (Debugging):** Análise de waveforms e logs.
- **Análise de Desempenho:** Interpretação das métricas de CPI.
- **Redação Técnica:** Estruturação e formatação deste documento.

8.2 Prompts Baseados em Spec-Driven Development (SDD)

Prompt 1: Implementação da Unidade de Branch (Parte 1)

> ***Contexto:*** *Estou desenvolvendo um processador RISC-V de 5 estágios em Verilog.* ***Problema:*** *Preciso implementar a unidade de controle de branch (`BranchUnit.v`) para tratar a instrução `BEQ`.* ***Requisitos Funcionais:*** *1. Receber dois operandos de 32 bits do estágio EX. 2. Ativar `branch_taken` se forem iguais. 3. Calcular `branch_target`.* ***Restrições:*** *Código puramente combinacional.*

Prompt 2: Análise de Dependências e Inserção de NOPs (Parte 2)

> ***Contexto:*** *Pipeline RISC-V sem hardware de forwarding.* ***Problema:*** *Falhas RAW nos registradores `x2`, `x3`, `x4` e `x7`.* ***Especificação:*** *Determinar a quantidade de NOPs necessários e recalcular o offset do `BEQ`.* ***Restrição:*** *Solução estritamente via software.*

Prompt 3: Implementação de Forwarding e Hazard Detection (Parte 3)

> ***Contexto:*** *Finalização do hardware do pipeline RISC-V.* ***Problema:*** *Implementar `ForwardingUnit.v` e `HazardDetectionUnit.v`.* ***Requisitos:*** *Detectar dependências*

> *RAW e hazards load-use; gerar sinais de `forward` e `stall`. **Restrições:** Seguir a arquitetura de Patterson & Hennessy.*

Prompt 4: Consolidação de Resultados e Relatório

> ***Contexto:** Resultados coletados das Partes 1, 2 e 3. **Tarefa:** Elaborar a análise de desempenho comparando as soluções de software e hardware, justificando o CPI e os stalls/bypasses.*